Qt5: Console Applications and Networking

Continuing with the series on Qt5 programming, this article takes the reader on to writing code and building a console application, which is also a network server.

In the article carried in the February 2015 issue of *OSFY*, we looked at how Qt makes programming easier by creating a whole new paradigm with extensions to a venerable programming language, and a code generator to help us out. In this article, let's start writing code, beginning with a console application, which is also a network server.

Getting started

What we'll be building is a 'fortune server' (the kind of 'fortune' you can expect to read in a fortune cookie!), which will select and send a random 'fortune' from a set of fortunes every time we connect to it and then disconnect.

Wait a minute, you say. Isn't Qt for GUI programming? Well, as I'd mentioned in Part 1 of this series of articles, Qt is an application framework, and while it has one of the industry's best GUI tool kits, that is only a part (albeit a big part) of what Qt does. Qt has an extremely robust network I/O module. We have the freedom to not use the GUI module but instead build a CLI application.

First install the Qt5 Core development packages, a C++ compiler, GNU Make and QtCreator. Figure 1 shows what you should be looking at when you start QtCreator.

Let's start by creating a project. On the top menu bar, select *File->New File Or Project*, hit *Ctrl+N* on the keyboard, or just click the big *New Project* button on the top left corner of the welcome screen. Either way, you'll end up looking at the dialogue box shown in Figure 2.

We're building the server now, so let's select *Applications* on the left under *Projects*, and *Qt Console Application* in the middle column. Once you're done, hit *Choose*. You'll then end up at the dialogue box shown in Figure 3.

Type in a name (I've called it FortuneServer), and hit *Next*. There's nothing to do in the *Kits* screen (just make sure *Desktop* is ticked), so hit *Next* again. In the next screen, you can add the project to version control, but since we haven't set up Git yet, just let that be. Hit *Finish*.

You should now be staring at the editor with a *main.cpp* file open. The contents of the file should be:

```
#include <QCoreApplication>

int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);
    return a.exec();
}
```

On the top left, you'll see a list of files that are part of the project. On the bottom left, you'll see a list of files that are open in the editor. Right now, that should just be *main.cpp*.